

모두를 위한 딥러닝 (이미지처리편)

경북대학교 IT교육센터

허 찬

차시(모듈)	차시별 주제	학습내용
1	딥러닝을 이용한 이미지 처리	이미지 처리의 이해 다양한 이미지처리의 응용 분야에 대한 이해
2	특징 추출의 이해	이미지 데이터 전처리 기법의 이해 이미지 특징 추출의 이해 및 실습 딥러닝을 이용한 학습 및 추론 과정의 이해
3	Convolution Neural Network	합성곱 신경망의 이해
4	CNN 모델의 이해	다양한 CNN 모델의 이해 딥러닝의 여러가지 요소 이해
5	CNN을 이용한 분류 모델 실습	CNN을 이용한 이미지 분류 모델 실습
6	전이 학습 모델의 이해	전이 학습의 이해 전이학습을 이용한 이미지 처리 모델 실습
7	객체 인식 모델의 이해	다양한 객체 인식 모델의 이해 Semantic segmentation (U-Net)의 이해
8	Transformer 기반 이미지 처리 모델 1	Attention과 Transformer의 이해
9	Transformer 기반 이미지 처리 모델 2	Transformer 기반 이미지 처리 모델 (ViT)의 이해
10	이미지 생성 모델의 이해	최신 이미지 생성 모델의 이해 오토인코더의 이해 및 실습
11	최신 이미지 처리 기술의 이해	최신 컴퓨터비전 모델의 이해 비디오 처리의 이해

CNN을 이용하는 다양한 task

Classification



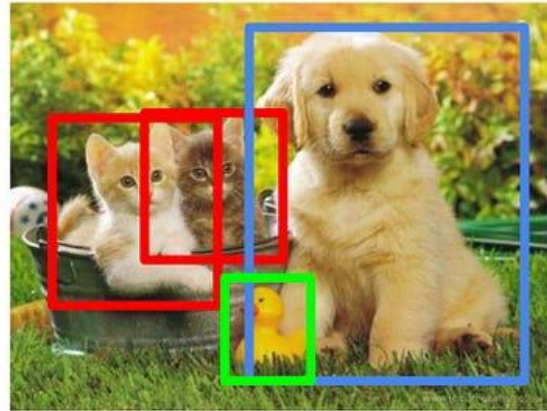
CAT

**Classification
+ Localization**



CAT

Object Detection



CAT, DOG, DUCK

**Instance
Segmentation**

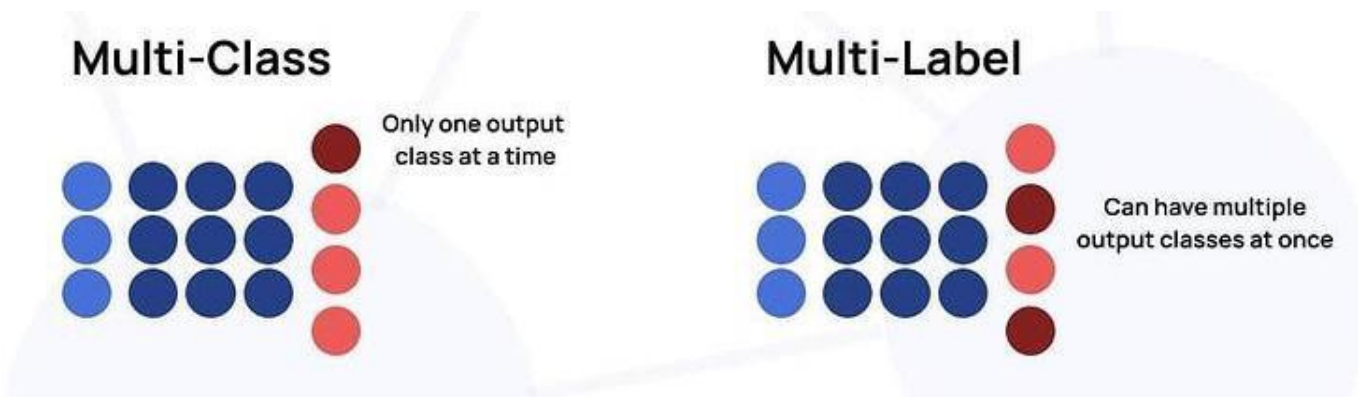


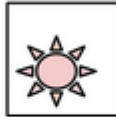
CAT, DOG, DUCK

Single object

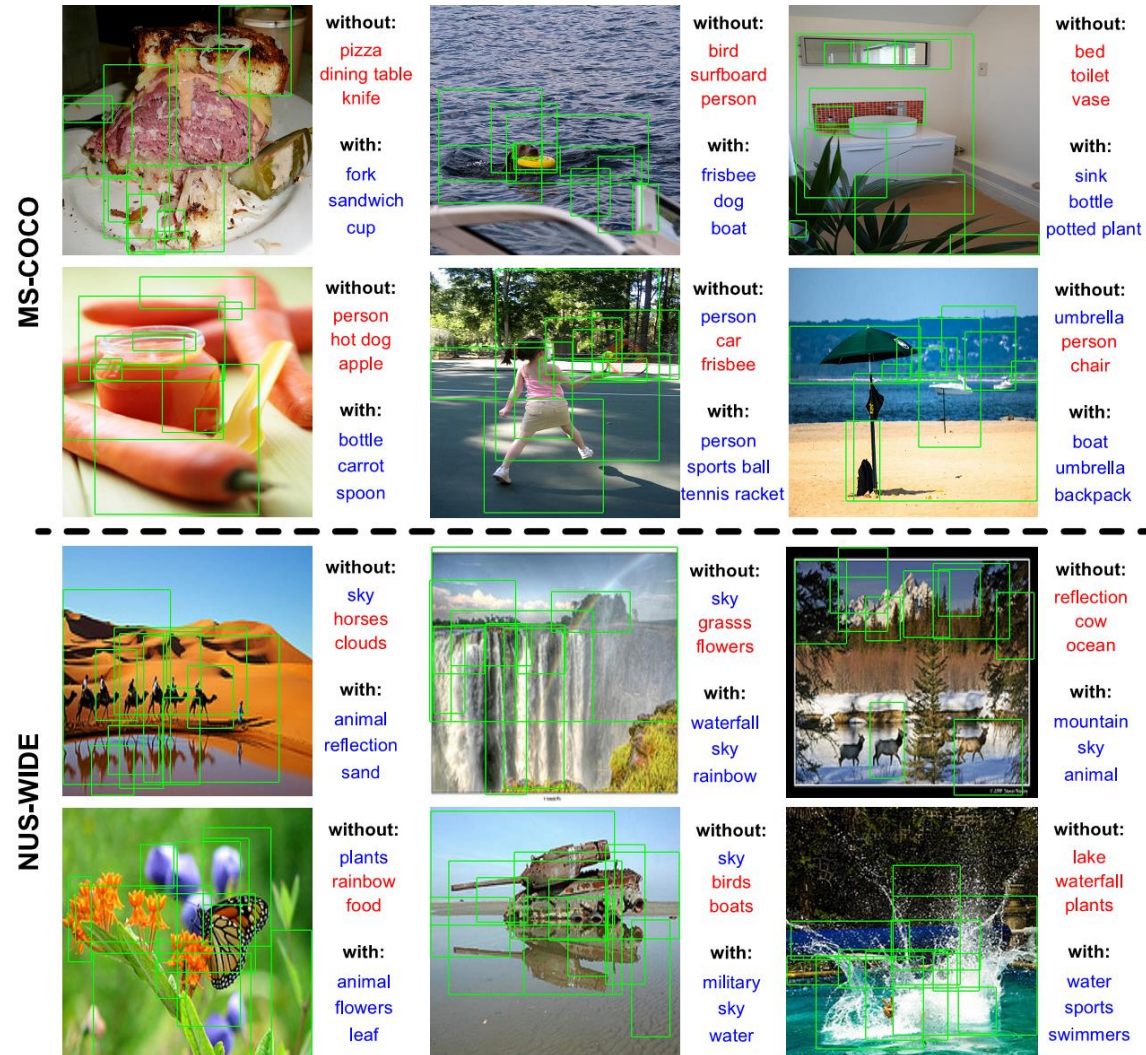
Multiple objects

7.1 CNN을 이용한 multi-label classification 문제



Multi-Class		Multi-Label	
C = 3	Samples	Samples	
	  	  	
	Labels (t) [0 0 1] [1 0 0] [0 1 0]	Labels (t) [1 0 1] [0 1 0] [1 1 1]	

7.1 CNN을 이용한 multi-label classification 문제



7.1 CNN을 이용한 multi-label classification 문제

Task 설명

- 목적: 하나의 그림에 대하여 multi-label image classification

Input:

- 자연 그림으로 이루어진 이미지 데이터

Output:

- 5개의 multi-label class



Miml dataset

<https://www.kaggle.com/datasets/twopothhead/miml-image-data>

Multi-instance multi-label classification under natural scene

7.1 CNN을 이용한 multi-label classification 문제

1) 데이터파일 압축해제

```
import zipfile  
zipfile.ZipFile('miml_dataset.zip').extractall()
```

7.1 CNN을 이용한 multi-label classification 문제

2) Module import

```
pip install git+https://github.com/keras-team/keras-preprocessing.git
```

```
from keras.models import Sequential
from keras_preprocessing.image import ImageDataGenerator
from keras.layers import Dense, Activation, Flatten, Dropout, BatchNormalization
from keras.layers import Conv2D, MaxPooling2D
from keras import regularizers, optimizers
import pandas as pd
import numpy as np
```


7.1 CNN을 이용한 multi-label classification 문제

3) 데이터셋 구성 및 전처리 (case1)

[illegible]

7.1 CNN을 이용한 multi-label classification 문제

3) 데이터셋 구성 및 전처리 (case2)

[illegible]

7.1 CNN을 이용한 multi-label classification 문제

4) 모델 구성

```
model = Sequential()
model.add(Conv2D(32, (3, 3), padding='same',
                 input_shape=(100,100,3)))
model.add(Activation('relu'))
model.add(Conv2D(32, (3, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))
model.add(Conv2D(64, (3, 3), padding='same'))
model.add(Activation('relu'))
model.add(Conv2D(64, (3, 3)))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.25))
model.add(Flatten())
model.add(Dense(512))
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(5, activation='sigmoid'))
model.summary()
```

7.1 CNN을 이용한 multi-label classification 문제

5) 모델 컴파일 및 학습

```
model.compile(optimizer='adam', loss="binary_crossentropy", metrics=["accuracy"])
model.fit(train_generator,
          steps_per_epoch=32,
          validation_data=valid_generator,
          validation_steps=32,
          epochs=1)
```

7.1 CNN을 이용한 multi-label classification 문제

6) 테스트 결과 출력

```
loss, test_accuracy = model.evaluate(valid_generator)
print(test_accuracy)

pred = model.predict(valid_generator,
                      steps=32,
                      verbose=1)

print(pred)
pred_bool = (pred > 0.5)
predictions = pred_bool.astype(int)

print(predictions)
columns = ["desert", "mountains", "sea", "sunset", "trees"]
# columns should be the same order of y_col
results = pd.DataFrame(predictions, columns=columns)
results["Filenames"] = valid_generator_filenames
ordered_cols = ["Filenames"] + columns
results = results[ordered_cols] # To get the same column order
results.to_csv("results.csv", index=False)
```

miml_labels_2.csv miml_labels_1.csv results.csv X ...

41 to 50 of 100 entries

Filenames	desert	mountains	sea	sunset	trees
1841.jpg	0	0	0	0	1
1842.jpg	0	0	0	0	0
1843.jpg	0	0	0	0	1
1844.jpg	0	0	0	0	1
1845.jpg	0	0	0	0	1
1846.jpg	0	0	0	0	1
1847.jpg	0	0	0	0	1
1848.jpg	0	1	0	0	0
1849.jpg	0	0	0	0	0
1850.jpg	0	0	0	0	1

Show 10 per page 1 2 3 4 5 6 7 8 9 10

7) 결과 그림 확인

```
import matplotlib.pyplot as plt
import matplotlib.image as img

img_test = img.imread('miml_dataset/images/1849.jpg')
plt.imshow(img_test)
plt.show()
```

7.2 Object detection?



7.2 Object detection?

1. 객체 탐지의 구성 요소

위치 검출(Localization):

- 객체의 위치를 파악하고, 객체를 감싸는 최소 크기의 사각형(바운딩 박스)을 결정.

분류(Classification):

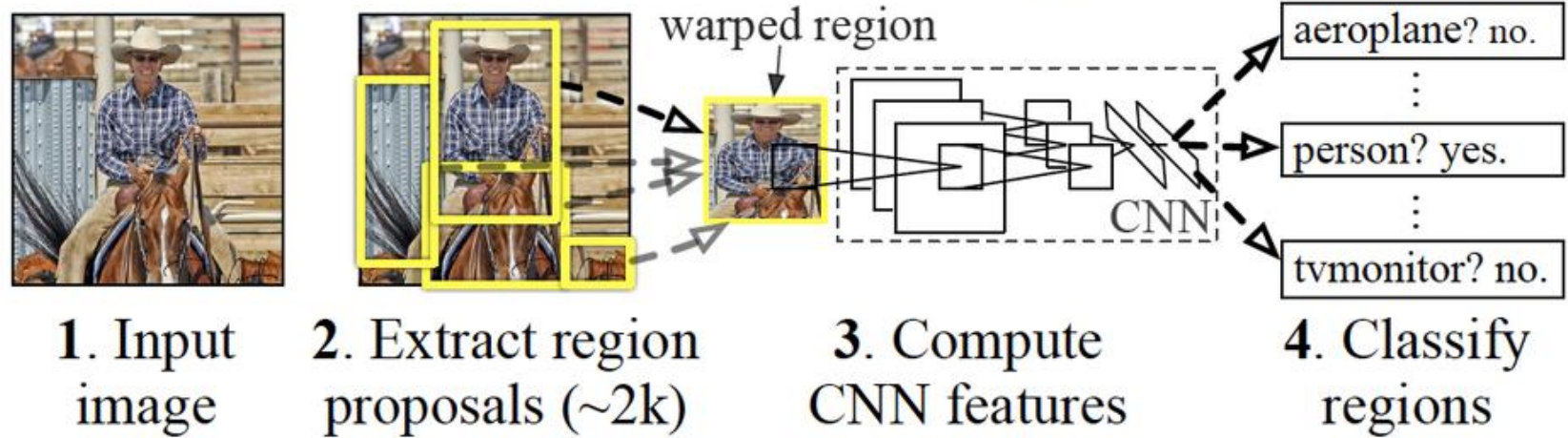
- 이미지에서 관측된 객체가 무엇인지 판단합니다. 예를 들어, 사람, 자동차, 나무 등을 결정.

2. 객체 인식의 응용

- **보안 감시:** 공공장소나 주요 시설에서 비정상적인 행동이나 위협을 감지.
- **자율 주행 차량:** 도로 위의 차량, 보행자, 신호등 등을 인식하여 안전하게 운행.
- **헬스케어:** 의료 영상에서 이상 현상을 탐지하거나 특정 조직을 식별.
- **산업 자동화:** 제조 라인에서 제품의 결함을 검출하거나 품질을 관리.

7.2 Object detection?

R-CNN: *Regions with CNN features*



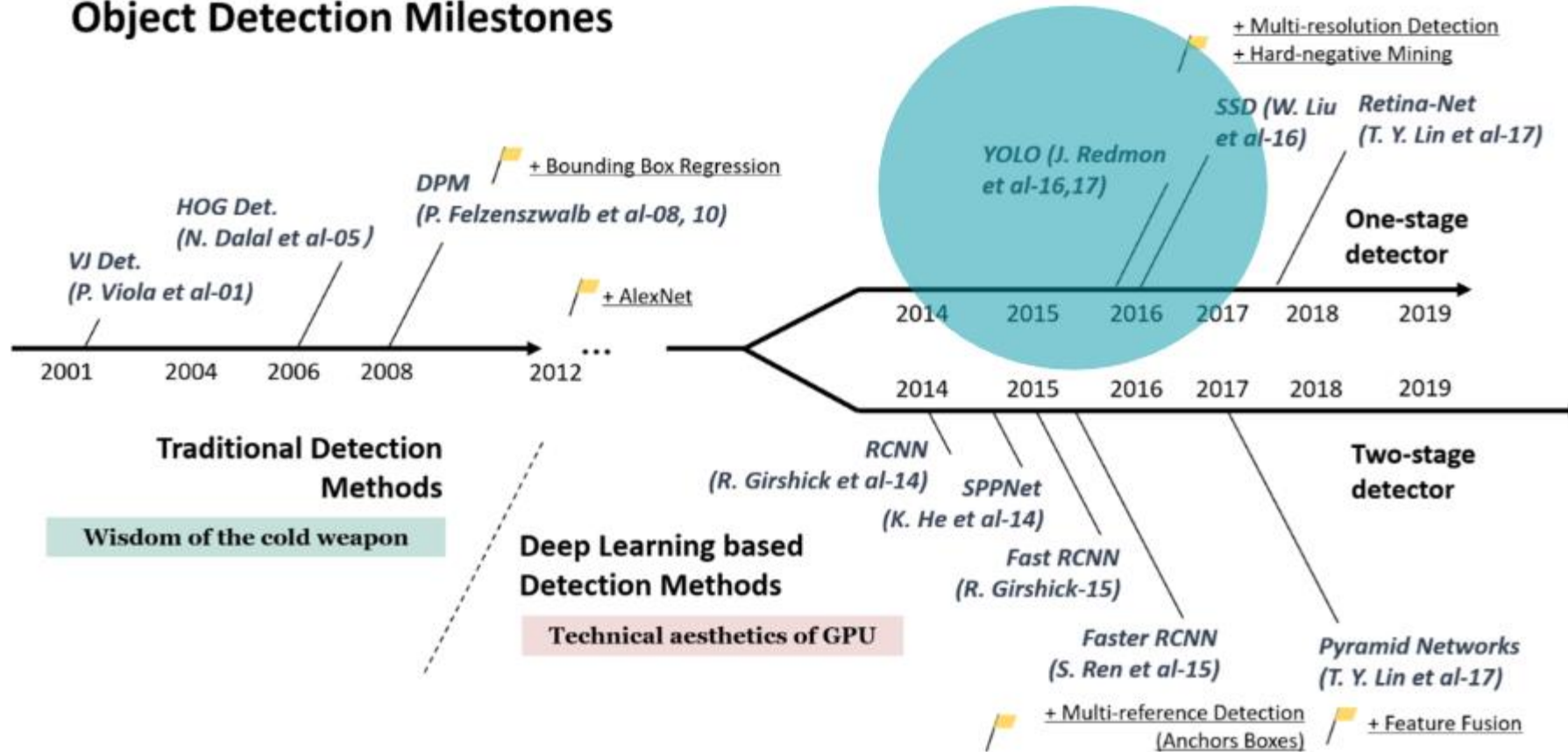
Task: →

1. Region Proposal

2. Region Classification

7.2 Object detection?

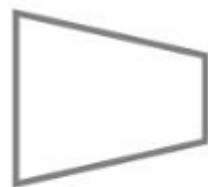
Object Detection Milestones



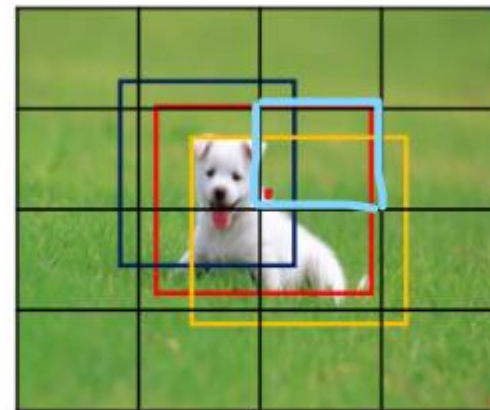
7.2 Object detection?



Input Image



Conv layers



-  Groundtruth
-  예측한 yellow bbox
-  예측한 blue bbox

7.2 Object detection?

단일 신경망 구조

- YOLO는 이미지 전체를 한 번에 보고, 여러 객체의 종류와 위치를 동시에 예측.
- 이러한 접근 방식은 이미지를 한 번만 처리함으로써 처리 속도를 대폭 향상시키며 실시간 처리에 적합.

그리드 시스템

- 입력 이미지를 $S \times S$ 그리드로 나누고, 각 그리드 셀이 객체의 중심을 포함.
- 각 셀은 바운딩 박스를 예측하고, 각 박스에 대해 확신(confidence) 점수를 제공.
- confidence score는 박스가 객체를 포함할 확률과 그 예측이 정확할 확률을 곱한 값.

클래스 확률

- 각 그리드 셀은 또한 C 개의 조건부 클래스 확률을 예측.
- 해당 셀이 특정 클래스의 객체를 포함할 조건 하에 계산된 확률.

속도와 정확도

- YOLO는 이미지를 전체적으로 한 번만 보기 때문에 처리 속도가 매우 빠름.
- 이는 비디오 스트림의 실시간 분석과 같은 응용에서 큰 장점을 제공.
- 빠른 속도 외에도 YOLO는 매우 높은 정확도를 보여주며, 특히 객체의 일반적인 위치를 감지하는 데 효과적.

7.2 Object detection?

YOLO의 세대와 발전

- **YOLOv1**: 초기 버전은 실시간 처리 속도를 강조하면서도 높은 정확도를 달성.
- **YOLOv2**: 더 많은 데이터와 새로운 기법을 도입하여 속도와 정확도를 모두 개선.
- **YOLOv3**: 더 깊은 네트워크와 3개의 다른 스케일에서의 예측을 도입하여 소형 객체 검출 성능을 향상.
- **YOLOv4, YOLOv5**: 최신 버전에서는 성능 최적화와 함께 여러 최신 기술을 통합하여 사용 편의성과 정확도 향상
- ...

7.2 Object detection model

```
pip install git+https://github.com/keras-team/keras-preprocessing.git
```

```
!pip install tensorflow==2.8.0
```

```
!pip install yolov3_tf2
```


7.2 Object detection model

```
import requests
import os
import tensorflow as tf
import numpy as np
import cv2
import matplotlib.pyplot as plt
from yolov3_tf2.models import YoloV3
from yolov3_tf2.utils import load_darknet_weights
from yolov3_tf2.utils import draw_outputs

# 파일 다운로드 함수
def download_file(url, dest):
    response = requests.get(url, stream=True)
    if response.status_code == 200:
        with open(dest, 'wb') as f:
            for chunk in response.iter_content(1024):
                f.write(chunk)
        print(f"Downloaded {url} to {dest}")
    else:
        print(f"Failed to download {url}")

# yolov3.weights 파일 다운로드
weights_url = 'https://github.com/patrick013/Object-Detection---Yolov3/blob/master/model/yolov3.weights?raw=true'
weights_path = 'yolov3.weights'

if not os.path.exists(weights_path):
    download_file(weights_url, weights_path)
```

- 학습 x. 가중치를 가지고 와서 추론에만 사용

7.2 Object detection model

```
# COCO 클래스 이름 파일 준비
coco_names_url =
'https://raw.githubusercontent.com/pjreddie/darknet/master/data/coco
.names'
coco_names_path = 'coco.names'

if not os.path.exists(coco_names_path):
    download_file(coco_names_url, coco_names_path)

# YOLOv3 모델 로드
yolo = YoloV3(classes=80)
load_darknet_weights(yolo, weights_path)

# COCO 클래스 이름 로드
class_names = [c.strip() for c in open(coco_names_path).readlines()]
```

7.2 Object detection model

이미지 로드 및 전처리

```
def load_image(image_path):  
    img = cv2.imread(image_path)  
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  
    img = cv2.resize(img, (416, 416))  
    img = img / 255.0  
    return img[np.newaxis, ...].astype(np.float32), img
```

객체 탐지 수행

```
def predict_objects(model, image):  
    boxes, scores, classes, nums = model(image)  
    return boxes, scores, classes, nums
```

결과 시각화

```
def draw_boxes(image, boxes, scores, classes, nums, class_names):  
    wh = np.flip(image.shape[0:2])  
    for i in range(nums[0]):  
        x1y1 = tuple((np.array(boxes[0][i][0:2]) * wh).astype(np.int32))  
        x2y2 = tuple((np.array(boxes[0][i][2:4]) * wh).astype(np.int32))  
        image = cv2.rectangle(image, x1y1, x2y2, (0, 255, 0), 2)  
        image = cv2.putText(image, '{}  
{:.4f}'.format(class_names[int(classes[0][i])], scores[0][i]),  
                        x1y1, cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)  
    return image
```

7.2 Object detection model

이미지 파일 경로

```
image_path = "example1.jpg"
```

객체 탐지 수행

```
image, orig_img = load_image(image_path)
```

```
boxes, scores, classes, nums = predict_objects(yolo, image)
```

```
result_img = draw_boxes(orig_img, boxes, scores, classes, nums, class_names)
```

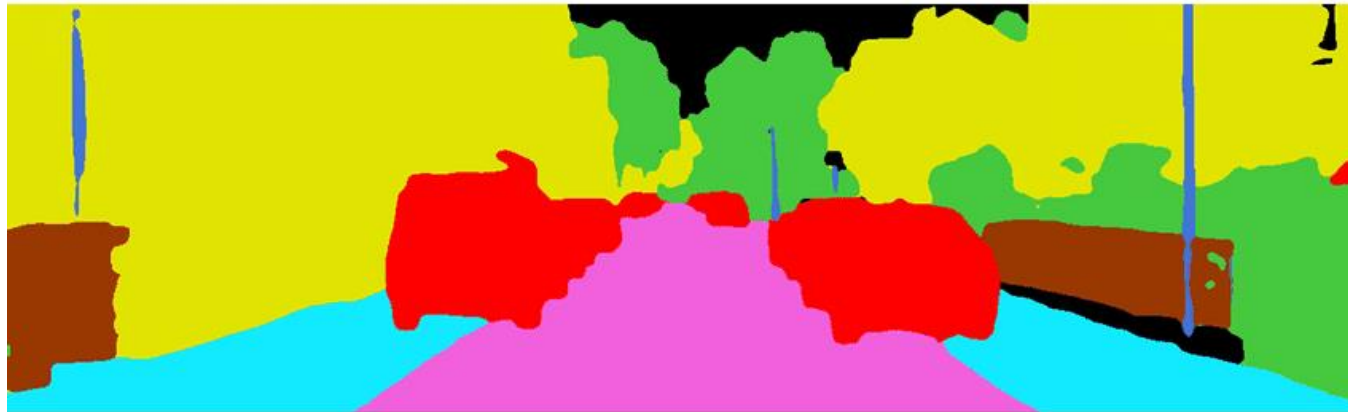
결과 시각화

```
plt.imshow(result_img)
```

```
plt.axis('off')
```

```
plt.show()
```

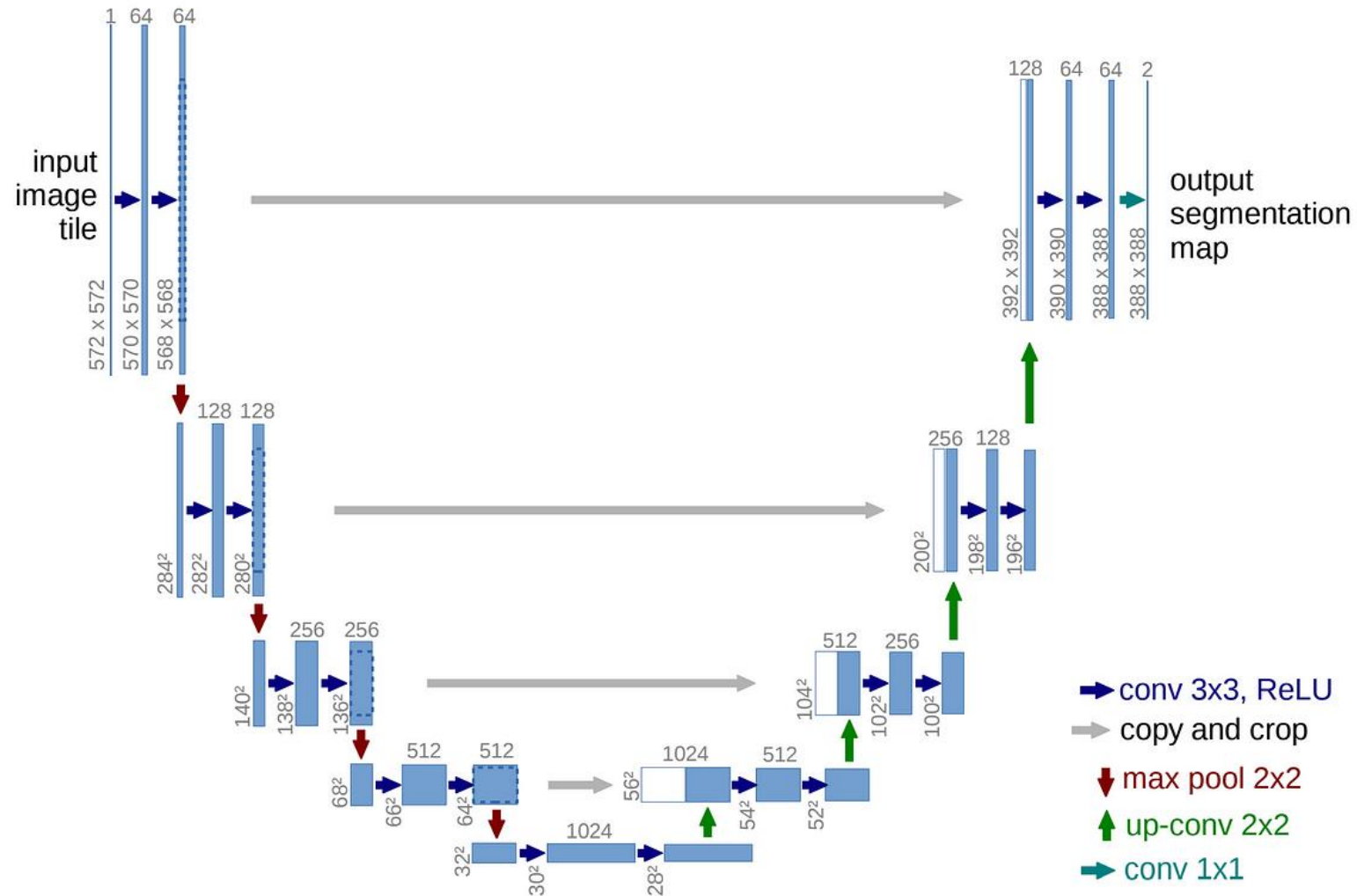
7.3 Semantic segmentation?



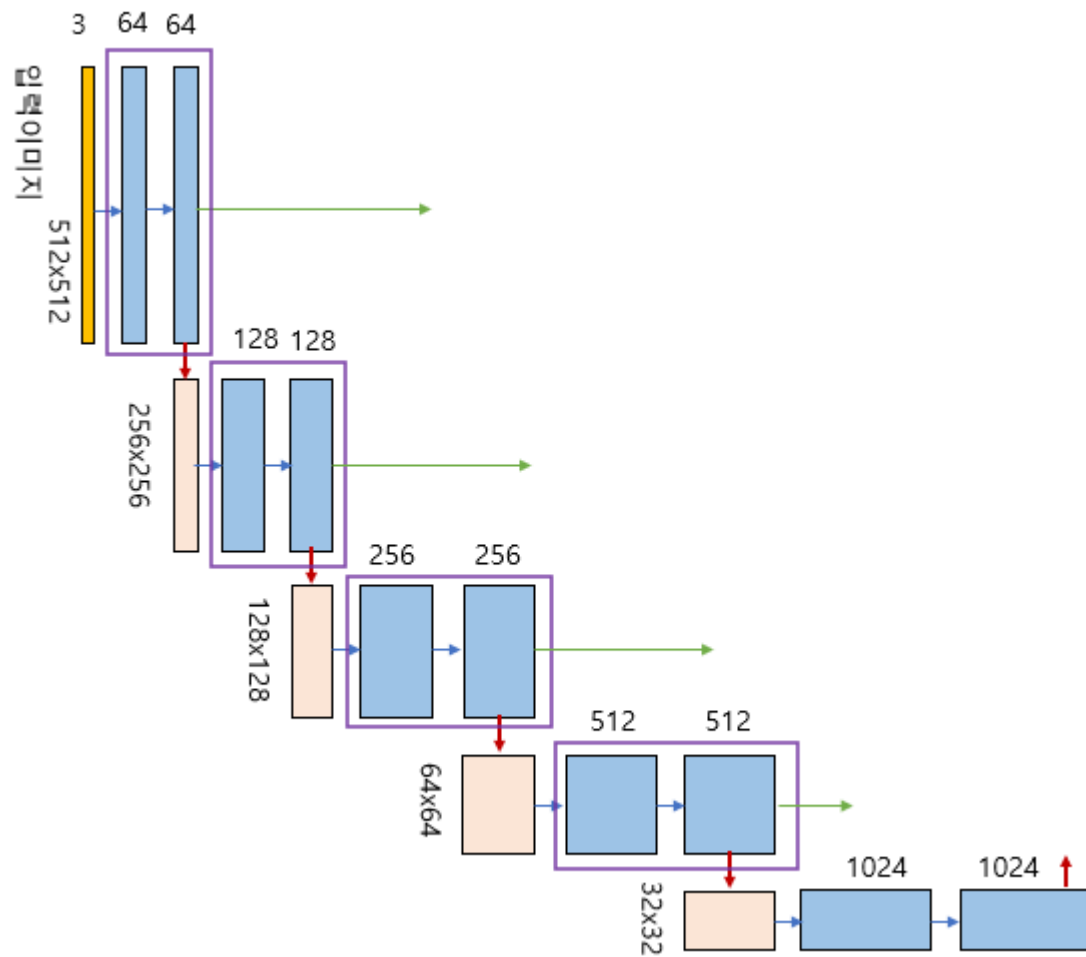
 Road	 Sidewalk	 Building	 Fence
 Pole	 Vegetation	 Vehicle	 Unlabel

정확한 픽셀별로 의미를 구분!

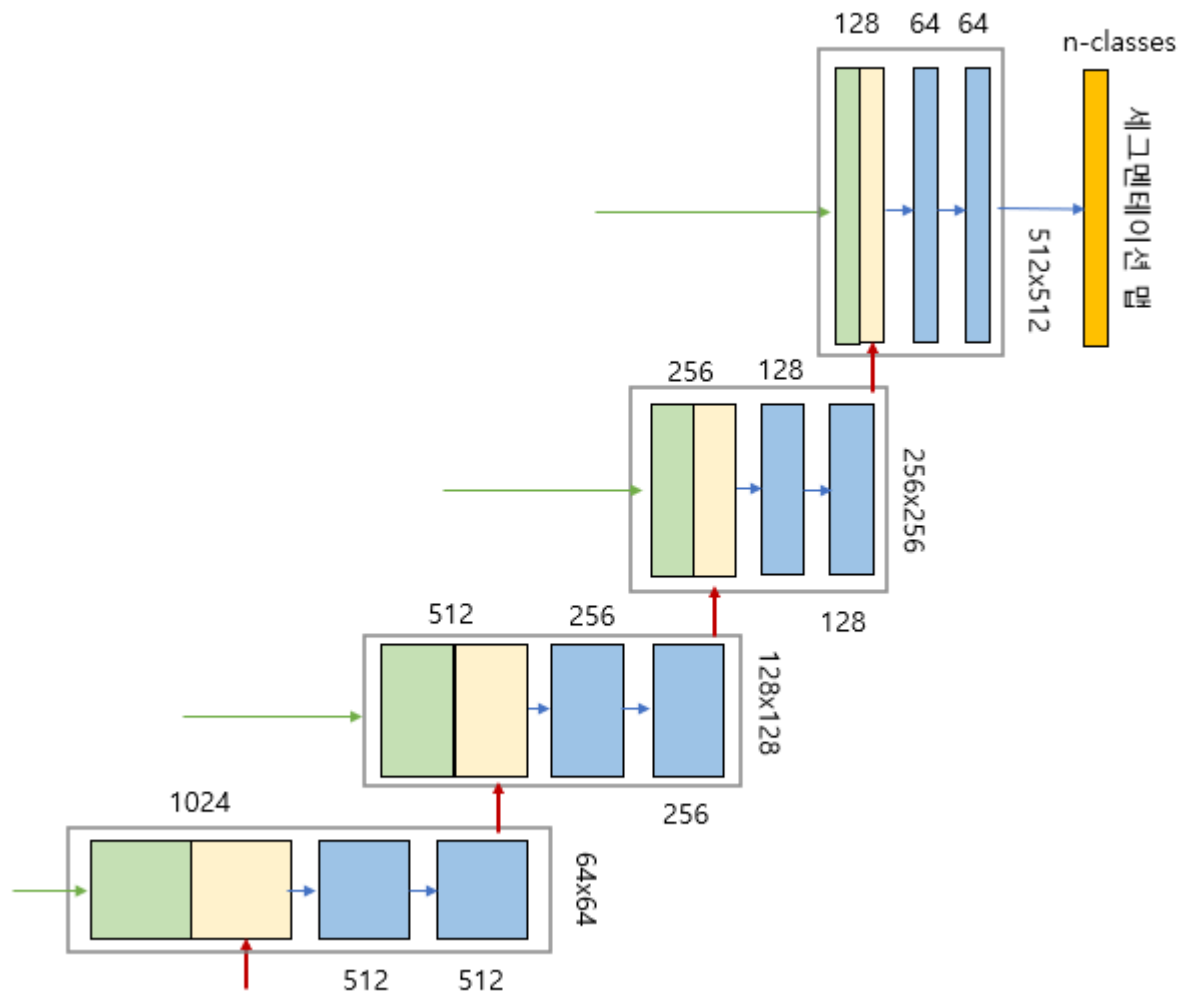
7.3 Semantic segmentation – U-Net



7.3 Semantic segmentation – U-Net



7.3 Semantic segmentation – U-Net



7.3 Semantic segmentation – U-Net

대칭 구조

- U-Net은 대칭적인 구조를 가지며, '인코더(Contracting)' 부분과 '디코더(Expanding)' 부분으로 구성.
- 인코더는 일반적인 컨볼루션 네트워크처럼 이미지에서 특징을 추출하는 역할을 하며, 여러 컨볼루션 레이어와 풀링 레이어를 통해 입력 이미지의 차원을 점점 줄임.
- 디코더는 이렇게 추출된 특징을 바탕으로 원래 이미지의 크기로 특징 맵을 점차 확장해 나가면서 세밀한 세분화를 가능하게 함.

스킵 연결 (Skip Connection)

- 인코더와 디코더 사이에는 스킵 연결이 있어, 인코더의 각 레벨에서의 특징 맵을 디코더의 해당 레벨과 연결.
- 이 스킵 연결은 디코더가 더 정확한 위치 정보를 복원하는 데 도움을 주어, 세분화의 정확도를 높임.

컨텍스트와 위치 정보의 복합 활용

- 인코더를 통해 얻은 컨텍스트 정보와 디코더를 통해 복원된 위치 정보가 결합되어, 세밀하고 정확한 semantic segmentation을 가능하게 함.

7.3 Semantic segmentation – U-Net

```
import tensorflow as tf
import tensorflow_datasets as tfds
import numpy as np
from tensorflow.keras import layers, models

# 데이터셋 로드
dataset, info = tfds.load('oxford_iiit_pet', with_info=True)
train_dataset = dataset['train']
test_dataset = dataset['test']

# 데이터셋 크기 계산
num_train_examples = info.splits['train'].num_examples
num_test_examples = info.splits['test'].num_examples

# 이미지와 마스크 전처리 함수
def normalize(input_image, input_mask):
    input_image = tf.cast(input_image, tf.float32) / 255.0
    input_mask = tf.cast(input_mask, tf.float32) / 255.0
    return input_image, input_mask

def load_image_train(datapoint):
    input_image = tf.image.resize(datapoint['image'], (128, 128))
    input_mask = tf.image.resize(datapoint['segmentation_mask'], (128, 128))
    input_image, input_mask = normalize(input_image, input_mask)
    return input_image, input_mask

def load_image_test(datapoint):
    input_image = tf.image.resize(datapoint['image'], (128, 128))
    input_mask = tf.image.resize(datapoint['segmentation_mask'], (128, 128))
    input_image, input_mask = normalize(input_image, input_mask)
    return input_image, input_mask
```

7.3 Semantic segmentation – U-Net

```
# 학습 데이터셋 준비
BATCH_SIZE = 32
BUFFER_SIZE = 1000
train = train_dataset.map(load_image_train, num_parallel_calls=tf.data.experimental.AUTOTUNE)
train_dataset = train.cache().shuffle(BUFFER_SIZE).batch(BATCH_SIZE).repeat()
train_dataset = train_dataset.prefetch(buffer_size=tf.data.experimental.AUTOTUNE)

test = test_dataset.map(load_image_test)
test_dataset = test.batch(BATCH_SIZE)

# steps_per_epoch 및 validation_steps 계산
steps_per_epoch = np.ceil(num_train_examples / BATCH_SIZE).astype(int)
validation_steps = np.ceil(num_test_examples / BATCH_SIZE).astype(int)

print("Steps per epoch:", steps_per_epoch)
print("Validation steps:", validation_steps)
```

7.3 Semantic segmentation – U-Net

```
import matplotlib.pyplot as plt

# 샘플 데이터 시각화
def display_sample_images(dataset, num_samples=5):
    plt.figure(figsize=(15, 15))
    for i, data in enumerate(dataset.take(num_samples)):
        input_image, input_mask = data
        plt.subplot(num_samples, 2, 2*i+1)
        plt.title('Input Image')
        plt.imshow(tf.keras.preprocessing.image.array_to_img(input_image))
        plt.axis('off')
        plt.subplot(num_samples, 2, 2*i+2)
        plt.title('Mask')
        plt.imshow(tf.keras.preprocessing.image.array_to_img(input_mask))
        plt.axis('off')
    plt.show()

# 학습 데이터셋에서 샘플 이미지와 마스크 시각화
display_sample_images(train)
```

7.3 Semantic segmentation – U-Net

```
import tensorflow as tf
from tensorflow.keras import layers, models

def small_unet_model(input_size=(128, 128, 3)):
    inputs = layers.Input(input_size)

    # 인코더 (Contracting Path)
    conv1 = layers.Conv2D(32, 3, activation='relu', padding='same')(inputs)
    conv1 = layers.Conv2D(32, 3, activation='relu', padding='same')(conv1)
    pool1 = layers.MaxPooling2D(pool_size=(2, 2))(conv1)

    conv2 = layers.Conv2D(64, 3, activation='relu', padding='same')(pool1)
    conv2 = layers.Conv2D(64, 3, activation='relu', padding='same')(conv2)
    pool2 = layers.MaxPooling2D(pool_size=(2, 2))(conv2)

    conv3 = layers.Conv2D(128, 3, activation='relu', padding='same')(pool2)
    conv3 = layers.Conv2D(128, 3, activation='relu', padding='same')(conv3)
    pool3 = layers.MaxPooling2D(pool_size=(2, 2))(conv3)

    # 중앙 (Bridge)
    conv4 = layers.Conv2D(256, 3, activation='relu', padding='same')(pool3)
    conv4 = layers.Conv2D(256, 3, activation='relu', padding='same')(conv4)

    # 디코더 (Expansive Path)
    up5 = layers.Conv2D(128, 2, activation='relu', padding='same')(layers.UpSampling2D(size=(2, 2))(conv4))
    merge5 = layers.concatenate([conv3, up5], axis=3)
    conv5 = layers.Conv2D(128, 3, activation='relu', padding='same')(merge5)
    conv5 = layers.Conv2D(128, 3, activation='relu', padding='same')(conv5)

    up6 = layers.Conv2D(64, 2, activation='relu', padding='same')(layers.UpSampling2D(size=(2, 2))(conv5))
    merge6 = layers.concatenate([conv2, up6], axis=3)
    conv6 = layers.Conv2D(64, 3, activation='relu', padding='same')(merge6)
    conv6 = layers.Conv2D(64, 3, activation='relu', padding='same')(conv6)

    up7 = layers.Conv2D(32, 2, activation='relu', padding='same')(layers.UpSampling2D(size=(2, 2))(conv6))
    merge7 = layers.concatenate([conv1, up7], axis=3)
    conv7 = layers.Conv2D(32, 3, activation='relu', padding='same')(merge7)
    conv7 = layers.Conv2D(32, 3, activation='relu', padding='same')(conv7)

    conv8 = layers.Conv2D(1, 1, activation='sigmoid')(conv7)

    model = models.Model(inputs=inputs, outputs=conv8)

    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

    return model

model = small_unet_model()
model.summary()
```

7.3 Semantic segmentation – U-Net

```
EPOCHS = 20

history = model.fit(train_dataset, epochs=EPOCHS, steps_per_epoch=steps_per_epoch,
                    validation_data=test_dataset, validation_steps=validation_steps)

import matplotlib.pyplot as plt

def display(display_list):
    plt.figure(figsize=(15, 15))

    title = ['Input Image', 'True Mask', 'Predicted Mask']

    for i in range(len(display_list)):
        plt.subplot(1, len(display_list), i+1)
        plt.title(title[i])
        plt.imshow(tf.keras.preprocessing.image.array_to_img(display_list[i]))
        plt.axis('off')
    plt.show()

for image, mask in test_dataset.take(1):
    sample_image, sample_mask = image, mask
    pred_mask = model.predict(sample_image)
    display([sample_image[0], sample_mask[0], pred_mask[0]])
```

감사합니다